10. Queues

First In, First Out — The Earliest Arrival Leaves First

Previous Section:

9. Stacks

Contents:

Have you ever waited in line to buy a ticket, order food, or board a bus?

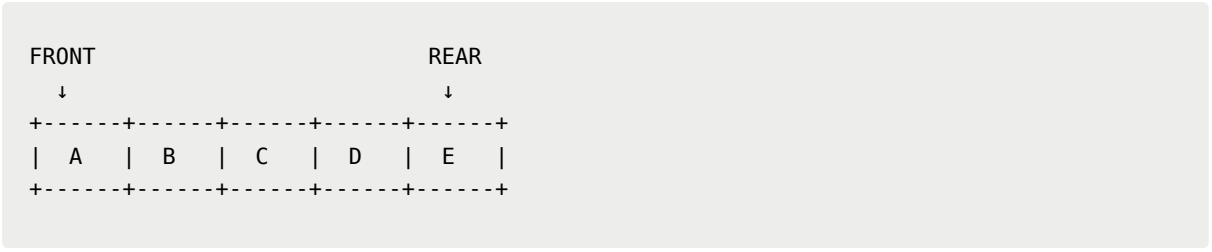
In everyday life, we naturally form a queue. The first person who joins the line is usually the first person to be served. As that person leaves, everyone behind moves one step closer to the front. New people continue to join at the back of the line.

Queues in computer science follow exactly the same idea. When we store data in a queue, the first element that enters the queue is the first element that

leaves it. New elements are always added at the rear, while existing elements are removed from the front.

Because of this behavior, queues are commonly used whenever we need to process tasks in the order they arrive.

A simple queue can be visualized as:



Element **A** entered first, so it will be removed first. Element **E** entered most recently, so it will be removed last.

1. What is a Queue?

A **Queue** is a linear data structure that follows the **First In First Out (FIFO)** principle. This means:

- The first element inserted into the queue is the first element removed.
- The most recently inserted element remains in the queue until all elements before it have been processed.

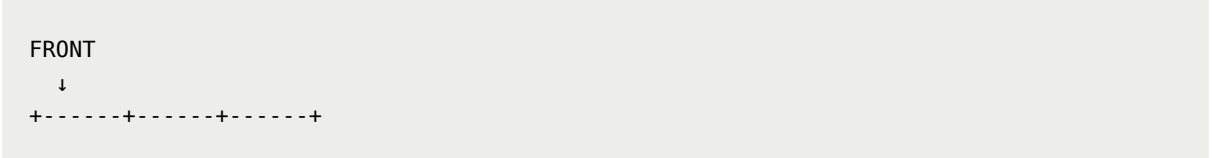
Unlike stacks, where insertion and deletion occur at the same end, queues separate these operations:

- **Insertion (Enqueue)** occurs at the rear.
- **Deletion (Dequeue)** occurs at the front.

We usually keep track of a queue using two special positions:

FRONT

The front points to the first element in the queue. This element has been waiting the longest and will be removed next.



```
|  A   |  B   |  C   |
+------+------+------+
```

REAR (also called BACK or TAIL)

The rear points to the most recently inserted element. New elements are always added after the rear.

```
               REAR
                ↓
+------+------+------+
|  A   |  B   |  C   |
+------+------+------+
```

Together:

```
FRONT            REAR
  ↓               ↓
+------+------+------+
|  A   |  B   |  C   |
+------+------+------+
```

As elements are added and removed, the positions of FRONT and REAR change accordingly.

2. Types of Queues

While the basic queue is the most common form, several specialized versions exist to solve different problems.

2.1. Linear Queue

A **Linear Queue** is the standard implementation of a queue. It contains two positions: FRONT and REAR. Elements are inserted from the rear and removed from the front.

```
FRONT                       REAR
  ↓                          ↓
+------+------+------+------+------+
```

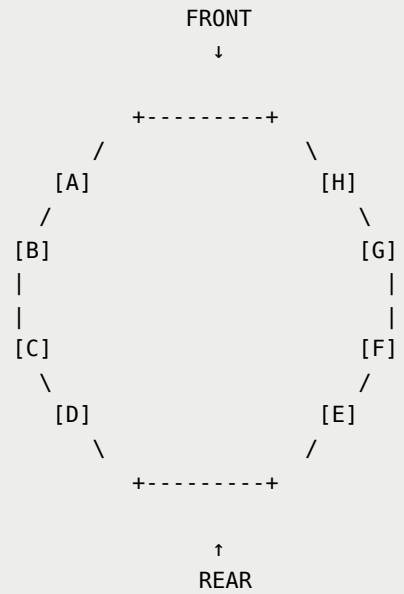
```
| A  | B  | C  | D  | E  |
+----+----+----+----+----+
```

This is the queue type most beginners encounter first.

2.2 Circular Queue

A **Circular Queue** connects the last position back to the first position.

Instead of treating the queue as a straight line, we treat it as a circle.



When the rear reaches the end, it can return to the beginning if empty spaces are available.

Advantages of circular queues include better memory utilization, reduced wasted space, more efficient queue management

Circular queues are commonly used in buffering systems and operating systems.

2.3. Double-Ended Queue (Deque)

A **Double-Ended Queue**, often shortened to **Deque**, allows insertion and deletion at both ends.

```
Front End                    Rear End
     ↓                          ↓
+------+------+------+------+------+
|  A   |  B   |  C   |  D   |  E   |
+------+------+------+------+------+
```

Operations can occur from either side depending on the implementation.

- **Input Restricted Deque:** Insertion can occur from only one end. Deletion can occur from either end.

```
Insert → [ Queue ] ← Delete
                     Delete →
```

- **Output Restricted Deque:** Insertion can occur from either end. Deletion can occur from only one end.

```
Insert → [ Queue ] ← Insert
Delete →
```

Deques are useful when we need more flexibility than a standard queue.

2.4. Priority Queue

A **Priority Queue** processes elements according to priority rather than arrival time. Elements with higher priority are processed first.

- **Ascending Priority Queue:** Smaller priority values are processed first.

```
Priority: 1  2  3  4  5
          ↑
      Removed First
```

- **Descending Priority Queue:** Larger priority values are processed first.

```
Priority: 5  4  3  2  1
          ↑
      Removed First
```

Priority queues are widely used in: Operating systems; Task scheduling; Pathfinding algorithms; Network routing.

Because of their importance, priority queues are usually studied as a separate topic.

3. Applications of Queues

Queues appear in many real-world and computing scenarios because they naturally organize tasks in the order they arrive.

- Multiprogramming: When multiple programs reside in memory at the same time, they must wait for processor time. These waiting programs are commonly organized as queues.

```
Program A → Program B → Program C → CPU
```

- **Computer Networks:** Routers and switches receive large numbers of packets every second. Incoming packets are stored in queues and processed one after another. Mail servers also use queues to manage outgoing and incoming messages.

```
Packet 1
Packet 2
Packet 3
    ↓
 Network Device
```

- **Job Scheduling:** Operating systems maintain queues of processes waiting to execute. The processor serves jobs according to scheduling rules.

```
Job 1
Job 2
Job 3
   ↓
 Processor
```

- **Shared Resources:** When multiple users need access to a single resource, a queue helps maintain fairness. Examples include printer queues, database requests, customer service systems

```
User A
User B
User C
   ↓
 Shared Resource
```

- **CPU and Device Communication:** Queues often act as buffers between devices operating at different speeds. Examples are keyboard and CPU, network devices, disk drives. The queue prevents data loss when one component works faster than another.

```
Fast Device → Queue → Slow Device
```

- **Breadth-First Search (BFS):** When exploring graphs or trees level by level, we use a queue to store nodes waiting to be visited. Without queues, Breadth-First Search would not be possible.

```
Level 1
   ↓
Level 2
   ↓
Level 3
```

Advantages of Queues

Queues offer several important benefits:

- Efficiently manage large amounts of data.
- Naturally process tasks in arrival order.
- Provide simple insertion and deletion operations.

- Useful when many users share the same service.
- Fast for inter-process communication.
- Can be used to implement other data structures and algorithms.
- Widely supported in programming languages and libraries.

Disadvantages of Queues

Despite their usefulness, queues also have limitations:

- Insertion or deletion in the middle is inefficient.
- Standard queues only allow insertion at the rear.
- Searching for a specific element requires scanning the queue sequentially.
- Array-based implementations require a predefined maximum size.
- Queue operations become less efficient if implemented poorly.

Summary

A **Queue** is a linear data structure that follows the **First In First Out (FIFO)** principle, where the earliest inserted element is the earliest removed element.

Every queue contains two important positions:

- **FRONT** — the first element to be processed.
- **REAR** — the most recently inserted element.

Several variations exist, including: Linear Queue; Circular Queue; Double-Ended Queue (Deque); Priority Queue

Queues are essential in operating systems, networking, scheduling, resource management, buffering, and graph traversal algorithms.

Their simple FIFO behavior makes them one of the most important data structures in computer science and software engineering.

References

https://drive.google.com/file/d/1A7WberDJxGmGtXpGZ7v-FFATGRIgKoQZ/view?usp=drive_link

https://www.geeksforgeeks.org/dsa/queue-data-structure/

https://www.programiz.com/dsa/queue

Next Section:

10.1. Operations on Queues